

Introduction to the Theory of Computation

Lecture 3. Problem Encodings

Neo (Yuan) Li

Sep 2026

Fudan University

yuan_li@fudan.edu.cn

Motivating Question

When we talk about problems like

Matrix Multiplication

Primality Testing

Shortest Path

what exactly are the **inputs** and **outputs** we are referring to? How do we encode problems as mathematical objects?

Motivating Question

When we talk about problems like

Matrix Multiplication

Primality Testing

Shortest Path

what exactly are the **inputs** and **outputs** we are referring to? How do we encode problems as mathematical objects?

A number n — in *binary*? Or in *decimal*?

A graph G — as an *adjacency list*? Or an *adjacency matrix*?

Two matrices A, B — row-major? Column-major?

Alphabets and Languages

Definition 1 (alphabet)

An *alphabet* is a set of symbols.

$\{0, 1\}$ is the **binary** alphabet.

$\{a, b, \dots, z\}$ is the **Roman** alphabet.

Definition 2 (string)

A *string* over an alphabet is a finite sequence of symbols from the alphabet.

The **empty string** is a string with no symbols, denoted by ϵ .

Examples. Let $\Sigma = \{0, 1\}$.

0, 1, 01, 101, 0000 are all strings over Σ .

ϵ is the empty string over Σ (it contains no symbols).

Definition 3

The set of all strings, including the empty string, over an alphabet Σ is denoted by Σ^* .

E.g., $\{0, 1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$.

Definition 4

The *length* of a string is its length as a sequence. Denote the length of a string w by $|w|$.

For example, $|\epsilon| = 0$, $|0| = 1$, $|0101| = 4$.

Definition 5

Denote by Σ^n the set of strings of length n .

So we have

$$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n.$$

Definition 6 (concatenation)

Two strings over the same alphabet can be combined to form a third by the operation of *concatenation*.

The concatenation of strings x and y is denoted by xy .

For example, if $x = 01$ and $y = 10$, then $xy = 0110$.

Definition 7 (language)

A *language* is a set of strings over an alphabet, denoted by $L \subseteq \Sigma^*$.

Let $\Sigma = \{0, 1\}$.

Then $\emptyset$, Σ^* , Σ are all languages.

For example, let

$$\text{Even} = \{0, 10, 100, 110, 1000, \dots\},$$

$$\text{Prime} = \{10, 11, 101, 111, 1011, \dots\},$$

$$\text{Palindrome} = \{0, 00, 11, 101, 111, 010, \dots\}$$

are all languages.

Definition 8 (complement)

Let $L \subseteq \Sigma^*$. The *complement* of L , denoted by $\bar{L}$, is $\Sigma^* \setminus L$.

Clearly, $\bar{\bar{L}} = L$.

Since languages are sets, the usual set operations apply: union ($\cup$), intersection ($\cap$), and difference ($\setminus$).

Definition 9

If L_1, L_2 are languages, the *concatenation* of L_1 and L_2 is defined by

$$L_1L_2 = \{w \in \Sigma^* : w = xy \text{ for some } x \in L_1, y \in L_2\}.$$

For example, if $L_1 = \{0, 1\}$ and $L_2 = \{00, 11\}$, then

$$L_1L_2 = \{000, 011, 100, 111\}.$$

Definition 10 (Kleene star)

Let $L \subseteq \Sigma^*$. The *Kleene star* of L , denoted by L^* , is the set of strings obtained by concatenating zero or more strings from L . That is,

$$L^* = \{w \in \Sigma^* : w = w_1 \cdots w_k$$

for $k \geq 0$ and $w_1, \dots, w_k \in L\}.$

Definition 11 (Kleene plus)

We write L^+ for the language LL^* . Equivalently,

$$L^+ = \{w \in \Sigma^* : w = w_1 \cdots w_k \\ \text{for } k \geq 1 \text{ and } w_1, \dots, w_k \in L\}.$$

For example, if $L = \{0, 1\}$, then

$$L^+ = \{0, 1, 00, 01, 10, 11, 000, \dots\} = \Sigma^* \setminus \{\epsilon\}.$$

Exercise

Let $\Sigma = \{0, 1\}$, $L_1 = \{0, 1\}$, $L_2 = \{00, 11\}$, $L_3 = \{1\}$.

Compute the following.

1) $L_2 L_3$

2) L_2^*

3) L_3^+

4) $\{\epsilon\}^*$

5) All strings of length at most 2 in L_1^*

Problem Encodings

Decision problems

A **decision problem** asks a yes/no question about the input.

A decision problem can be represented as a language $L \subseteq \Sigma^*$, where the answer for input x is **yes** if $x \in L$, and **no** if $x \notin L$.

For example,

(Primality testing) Given $n \in \mathbb{N}$, is n a prime?

(Hamiltonian cycle) Given a graph G , does G have a Hamiltonian cycle?

Computation problems

A **computation problem** asks to compute some output from the input, not just a yes/no answer.

A computation problem can be represented as a (partial) function

$$f : \Sigma^* \rightarrow \Sigma^* \cup \{\text{undefined}\}.$$

For example,

(Matrix multiplication) Given matrices A, B , compute AB .

(Determinant) Given a matrix A , compute $\det(A)$.

Let $f : \Sigma^* \rightarrow \Sigma^*$. Define the language

$$L_f = \{(x, i, b) : \text{the } i\text{-th bit of } f(x) \text{ is } b\}.$$

We can compute $f(x)$ by making $O(|f(x)|)$ queries to an oracle for L_f — one query per bit of $f(x)$.

So solving the decision problem L_f lets us solve the computation problem f .

Search problems

A **search problem** asks to find a solution satisfying some property.

Formally, given $x \in \{0, 1\}^*$, find $y \in \{0, 1\}^*$ such that $R(x, y)$ holds, where R is a relation.

For example,

(Hamiltonian cycle, search version) Given a graph G , find a Hamiltonian cycle in G , if one exists.

(Integer factoring) Given $n \in \mathbb{N}$, find a nontrivial factor of n .

Given $x \in \{0, 1\}^*$, we want to find $y \in \{0, 1\}^*$ such that $R(x, y)$ holds.

Define the language

$$L_R = \{(x, y_p) : \exists y \text{ s.t. } y_p \text{ is a prefix of } y \text{ and } R(x, y)\}.$$

If L_R can be decided, we can find y by querying L_R $O(|y|)$ times — extending the prefix y_p one bit at a time.

Optimization problems

An **optimization problem** asks to find a solution that maximizes or minimizes some objective.

Formally, $\text{OPT}(x) = \max\{c(x, y) : R(x, y)\}$, where R is a relation and c is a cost function.

For example,

(Max-SAT) Given a CNF formula, find an assignment satisfying as many clauses as possible.

(Traveling salesman) Given a weighted graph and a vertex s , find a shortest cycle starting and ending at s that visits every vertex.

Recall $\text{OPT}(x) = \max\{c(x, y) : R(x, y)\}$. Define the language

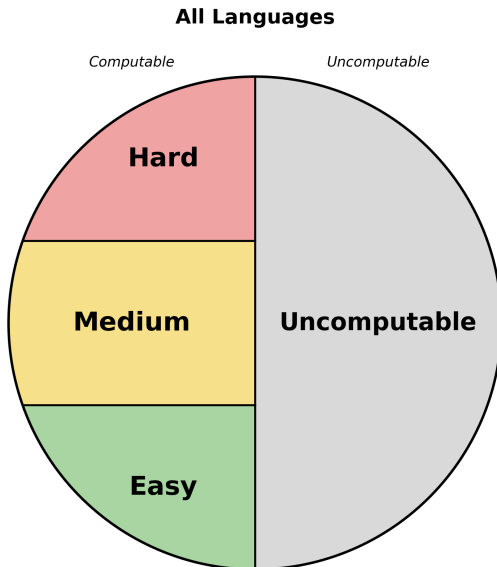
$$L_{\text{OPT}} = \{(x, k) : \exists y \text{ s.t. } R(x, y) \text{ and } c(x, y) \geq k\}.$$

By binary search, $\text{OPT}(x)$ can be computed by making $O(|\text{OPT}(x)|)$ queries to L_{OPT} .

Computation, search, and optimization problems can all be **reduced** to (a suitably defined) decision problem.

Without loss of generality, we study the computability and complexity of decision problems — equivalently, the computational complexity of **languages**.

The Landscape of Languages

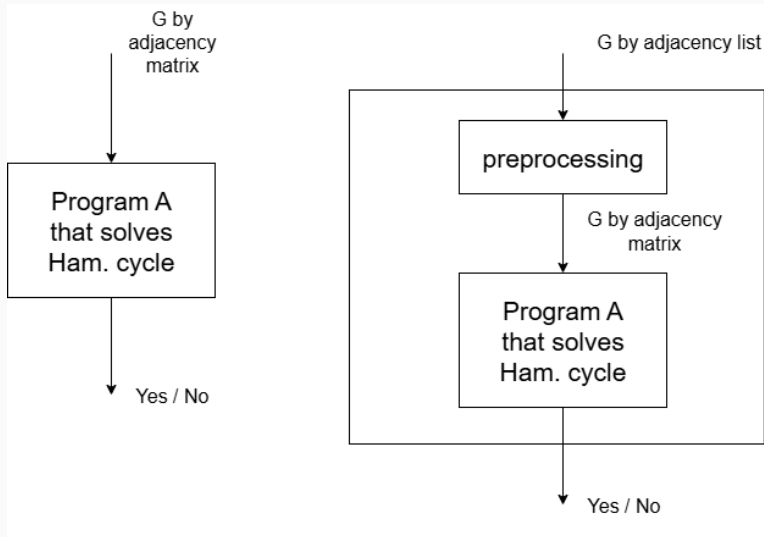


Switching Between Encodings

A graph may be represented by an **adjacency list** or **matrix**, and then encoded as a string using ASCII.

With preprocessing, we can switch between different encodings.

Switching encodings via preprocessing



A cautionary example: unary vs. binary

Consider **primality testing**: given n , is n a prime?

If n is written in **binary**, the input has length $O(\log n)$.

If n is written in **unary** (a string of n ones), the input has length n .

Converting unary to binary is efficient, but converting binary to unary **blows up exponentially** in length.

We only consider encodings that are **polynomial-time interconvertible** — i.e., converting between them takes polynomial time (and hence polynomial blow-up in length).

Under this restriction, the choice of encoding does **not** affect the complexity class a problem belongs to.